

Why convertToPlainObject Is Used

Explaining getOrderById, the Order Prisma model, and JSON.parse(JSON.stringify(...))

The Code in Question

getOrderById (Server Action):

```
export async function getOrderById(orderId: string) {
  return convertToPlainObject(
    await prisma.order.findFirst({
      where: { id: orderId },
      include: {
        orderitems: true,
        user: { select: { name: true, email: true } },
      },
    })
  );
}
```

Order model (schema.prisma):

```
model Order {
  id          String      @id @default(uuid()) @db.Uuid
  userId      String      @db.Uuid
  orderItems  Json         @db.Json()
  shippingAddress Json     @db.Json()
  paymentMethod String
  paymentResult Json?      @db.Json()
  itemsPrice  Decimal      @db.Decimal(12, 2)
  shippingPrice Decimal    @db.Decimal(12, 2)
  taxPrice    Decimal      @db.Decimal(12, 2)
  totalPrice  Decimal      @db.Decimal(12, 2)
  isPaid      Boolean      @default(false)
  paidAt      DateTime?    @db.Timestamp(6)
  isDelivered Boolean      @default(false)
  deliveredAt DateTime?    @db.Timestamp(6)
  createdAt   DateTime     @default(now()) @db.Timestamp(6)
  user        User         @relation(fields: [userId], references: [id], onDelete: Cascade)
  orderitems  orderItem[]
}
```

convertToPlainObject (helper):

```
export function convertToPlainObject<T>(data: T): T {
  return JSON.parse(JSON.stringify(data))
}
```

The Short Answer

It exists to strip out special JavaScript object types that Next.js cannot send from a Server Component / Server Action to a Client Component. Only plain data (strings, numbers, plain objects, arrays) can cross that boundary.

Analogy

Think of the server-to-client boundary as a border crossing. Only plain, “declared” cargo is allowed through — strings, numbers, plain objects, arrays. Some of the data coming back from Prisma is packed in special containers (class instances), and those get rejected at customs unless they are repacked into plain boxes first.

What's Actually in the “Special Containers”

1. **Decimal fields** — `itemsPrice`, `shippingPrice`, `taxPrice`, `totalPrice`. Prisma returns these as instances of a `Decimal` class (from `decimal.js`), not plain numbers. The class has methods on it, and Next.js's server-to-client serialization does not know how to send a class instance.
2. **DateTime fields** — `createdAt`, `paidAt`, `deliveredAt`. Prisma returns real JavaScript `Date` objects, another non-plain type for this boundary.
3. **Json fields** — `orderItems`, `shippingAddress`, `PaymentResult`. These are already plain, so they are not the issue on their own — but the whole object as a unit still fails if any other field on it is not plain.

What `convertToPlainObject` Actually Does

`JSON.parse(JSON.stringify(data))`

JSON.stringify turns every value into a plain string representation. A `Decimal` becomes a string like `"19.99"`; a `Date` becomes an ISO string like `"2026-09-25T10:00:00.000Z"`.

JSON.parse turns that string back into plain JS objects, arrays, strings, and numbers — with no class instances left.

Why It Shows Up Here Specifically

`getOrderById` is a Server Action fetching Prisma data that eventually gets passed to `OrderDetailsTable`, a component rendered with that data. If `OrderDetailsTable` (or something inside it) is a Client Component, passing the raw Prisma object across would throw a serialization error in Next.js. Wrapping the return value in `convertToPlainObject` fixes that.

The Tradeoff

After this conversion, `totalPrice` and the other `Decimal` fields become plain strings, not `Decimal` objects. That is exactly why the `OrderDetailsPage` code does `totalPrice: order.totalPrice.toString()` afterward — it is just making that conversion explicit and predictable, rather than relying on the implicit string coercion `JSON.stringify` already performed on the `Decimal`.

Note: the `.toString()` call is redundant

By the time `getOrderById` returns, `order.totalPrice` is already a plain string (Prisma's `Decimal` has a `toJSON()` method that `JSON.stringify` calls automatically, returning `.toString()` under the hood). So calling `.toString()` on it again in `OrderDetailsPage` is a no-op at runtime — it just returns the same string. It is still fine to keep: it costs nothing, documents intent, and protects against future changes to the data path. TypeScript still types the field as `Decimal` even after the runtime conversion strips the class away, which is why the call is there at all.